

# DATA SCIENCE LEARNING NOTES

## DAY 04 • FUNCTIONS IN PYTHON

**Student:** Muhammad Ahmad

**Goal:** Data Scientist

**Journey:** 60-Day Intensive Data Science Learning

### Day 4 Overview

Functions organize code into reusable blocks. They reduce repetition and make data processing logic easier to test, maintain, and reuse.

#### Learning Flow

def → function call → parameters → arguments → return → defaults → keyword arguments → scope → reusable Data Science functions

#### 1. Function Basics

A function is a reusable block of code for a specific task.

```
def greet():  
    print("Hello Muhammad")  
  
greet()
```

#### 2. Function with Parameter

A parameter lets a function receive input.

```
def greet(name):  
    print("Hello", name)  
  
greet("Ahmad")  
greet("Ali")
```

#### 3. Multiple Parameters

Functions can accept multiple inputs.

```
def student_info(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
student_info("Ahmad", 22)
```

#### 4. Parameters vs Arguments

Parameters are names in the definition. Arguments are actual values passed during the call.

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

#### 5. return Statement

return sends a result back to the caller.

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

#### 6. Average Function

Functions can package common analytical calculations.

```
def calculate_average(marks):
    total = sum(marks)
    return total / len(marks)

marks = [70, 80, 90]
print(calculate_average(marks))
```

## 7. Multiple Return Values

Python can return multiple values from one function.

```
def analyze_marks(marks):
    total = sum(marks)
    average = total / len(marks)
    highest = max(marks)
    lowest = min(marks)

    return total, average, highest, lowest

total, average, highest, lowest = analyze_marks([70, 80, 90, 60])
```

## 8. Default Parameters

A default value is used when an argument is omitted.

```
def greet(name="Student"):
    print("Hello", name)

greet()
greet("Ahmad")
```

## 9. Keyword Arguments

You can pass values using parameter names.

```
def student_info(name, age, cgpa):
    print(name, age, cgpa)

student_info(name="Ahmad", age=22, cgpa=3.23)
```

## 10. Function with Conditions

Functions can return decisions based on conditions.

```
def check_result(marks):
    if marks >= 50:
        return "Pass"
    return "Fail"

print(check_result(75))
print(check_result(40))
```

## 11. Grade Function

A reusable grading function keeps grade logic in one place.

```
def calculate_grade(marks):
    if marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 50:
        return "D"
    else:
        return "F"

print(calculate_grade(85))
```

## 12. Function with Loop

Functions and loops can process collections together.

```
def count_passed(marks):
    count = 0
```

```

for mark in marks:
    if mark >= 50:
        count += 1

return count

print(count_passed([45, 67, 80, 35, 90]))

```

### 13. List of Dictionaries

Functions can process structured records similar to simple tabular data.

```

def calculate_average_students(students):
    total = 0

    for student in students:
        total += student["marks"]

    return total / len(students)

```

### 14. Local and Global Scope

Variables created inside a function are local. Variables outside can be accessed from the function.

```

name = "Ahmad"

def greet():
    message = "Hello"
    print(message, name)

greet()

```

### 15. Docstrings

A docstring documents what a function does.

```

def calculate_average(numbers):
    """Return the average of numbers."""
    return sum(numbers) / len(numbers)

print(calculate_average([10, 20, 30]))

```

### 16. Practical Student Analysis

This combines parameters, calculations, conditions, and return values.

```

def analyze_student(name, marks):
    total = sum(marks)
    average = total / len(marks)

    if average >= 80:
        grade = "A"
    elif average >= 70:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 50:
        grade = "D"
    else:
        grade = "F"

    return name, total, average, grade

name, total, average, grade = analyze_student(
    "Ahmad", [85, 78, 92]
)

print(name, total, average, grade)

```

## DAY 4 PRACTICE

**Practice 1:** Create `calculate_square(number)` and return the square.

**Practice 2:** Create `calculate_average(numbers)` and test it with `[10, 20, 30, 40, 50]`.

**Practice 3:** Create `check_pass(marks)`. Return Pass for 50 or above, otherwise Fail.

**Practice 4:** Create `calculate_grade(marks)` using A, B, C, D, and F.

**Practice 5:** Create `analyze_marks(marks)` that returns total, average, highest, and lowest.

## DAY 4 MINI PROJECT

Build a Student Analysis System using reusable functions. Each student has a name and a list of subject marks.

```
students = [
{"name": "Ahmad", "marks": [85, 78, 92]},
{"name": "Ali", "marks": [72, 65, 80]},
{"name": "Usman", "marks": [45, 52, 40]}
]

def calculate_total(marks):
    return sum(marks)

def calculate_average(marks):
    return sum(marks) / len(marks)

def calculate_grade(average):
    if average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    return "F"

def analyze_student(student):
    name = student["name"]
    marks = student["marks"]
    total = calculate_total(marks)
    average = calculate_average(marks)
    grade = calculate_grade(average)
    return name, total, average, grade

for student in students:
    name, total, average, grade = analyze_student(student)
    print("Student:", name)
    print("Total:", total)
    print("Average:", round(average, 2))
    print("Grade:", grade)
    print()
```

## QUICK REVISION

```
def → define a function
parameter → input name in function definition
argument → actual value passed to function
return → send result back
default → fallback parameter value
keyword → argument passed by parameter name
local → variable inside function
global → variable outside function
docstring → function documentation
```

## PRINT VS RETURN

**print()** displays a value. **return** sends a value back so you can store, compare, or reuse it.

## DATA SCIENCE CONNECTION

Functions will become important for data cleaning, feature engineering, statistics, visualization, and Machine Learning workflows.

## DAY 4 COMPLETE

Next topic: Day 5, Python Lists and advanced list operations.